PL Project Documentation: Lexical Analyzer

Student Name: Jessica Mae Lanuzo
Course & Section: BSCS 3A
Date: May 15, 2026

1. Project Overview

This document details the implementation of a lexical analyzer (scanner) generated using Flex. The analyzer is designed to read source code written in Lua (5.4) and break it down into a formatted stream of valid tokens.

Supported Language Scope

The analyzer fully covers the following Lua 5.4 lexical elements:

- 19 reserved keywords (and, break, do, else, elseif, end, for, function, goto, if, in, local, not, or, repeat, return, then, until, while)
- Identifiers — any name beginning with a letter or underscore
- Numeric literals — decimal integers, IEEE-754 floats with optional exponents, and 0x-prefixed hexadecimal values
- String literals — single-quoted and double-quoted with full escape-sequence support
- Multi-line (long) strings delimited by [[and]]
- Boolean literals (true / false) and the nil literal
- All standard Lua operators including arithmetic, relational, bitwise, length (#), concatenation (..), and vararg (...)
- All Lua delimiters: parentheses, brackets, braces, comma, semicolon, colon, dot, and double-colon (::)
- Single-line comments (--) and block comments (--[[...]])

As it is mentioned, this analyzer is lexical only. **Syntactic or semantic errors are beyond its scope**, and only lexical invalidity, such as unrecognized characters and unterminated literals, is reported.

2. Token Specification

The following table outlines the lexical elements recognized by this analyzer and their corresponding Regular Expressions implemented in the Flex file.

Token Category	Description / Examples	Regular Expression
Keywords	Reserved words: if, else, function, local, etc.	("and"|"break"|"do"|"else"|"elseif"|"end"|"for"|"function"|"goto"|"if"|"in"|"local"|"not"|"or"|"repeat"|"return"|"then"|"until"|"while")
Identifiers	Variable or function names	[a-zA-Z_][a-zA-Z0-9_]*
Digits	One-digit numbers	[0-9]
Integers	Whole numbers	{DIGIT}+
Floats	Decimal/scientific notation numbers	{DIGIT}+\.{DIGIT}*{EXPONENT}?|{DIGIT}+{EXPONENT}|\.{DIGIT}+{EXPONENT}?
Hex	Hexadecimal integer literals	0[xX]{HEXDIGIT}+
Double Quoted Strings	"hello\n"	\"(\\.|[^\"\\])*\"
Single Quoted Strings	'world'	\'(\\.|[^\'\\])*\'
Operators	Arithmetic, relational, logical, bitwise operators	("..."|".."|"//"|"<<"|">>"|"=="|"~="|"<="|">="|"+"|"-"|"*"|"/"|"%"|"^"|"#"|"&"|"|"|"~"|"<"|">"|"=")
Delimiters	Structural punctuations	("::"|"("|")"|"["|"]"|"{"|"}"|","|";"|":"|".")

While not included in the Regular Expression Definitions, the lexer includes rules that allows it to identify comments (line and block), long strings, Boolean literals, and nil literals.

3. Setup and Execution

Prerequisites

To compile and run this project, ensure the following are installed:

- Flex (Fast Lexical Analyzer Generator)
- A C Compiler (e.g., GCC)

Build Instructions

Execute the following terminal commands to compile the analyzer:

```
# 1. Generate the C source file from the Flex file
flex lua_lexer.l

# 2. Compile the generated C file
gcc lex.yy.c -o lua_lexer

        # Note: -lfl is omitted because code defines custom yywrap
```

Execution

```
./lua_lexer input_code.lua
./lua_lexer             # type in input lines, press enter and Ctrl+D for EOF
```

4. Implementation Details

The Lua lexer is structured into **three sections** within the .l file: **Definitions** for C headers and regex macros, **Rules** for pattern–action pairs, and **User Code** for the main function and output logic. This clear separation makes the lexer easier to maintain and extend. State management relies on **Exclusive Start Conditions (%x)**, such as IN_BLOCK_COMMENT and IN_LONG_STRING, which allow the analyzer to correctly process block comments and long strings until the closing]] delimiter is found.

Whitespace is handled explicitly, with patterns like **[\t\r]+** ensuring spaces, tabs, and carriage returns are ignored during tokenization. Output is generated through the **print_token()** function, which uses ANSI color codes to produce a readable, tabular summary of tokens in the terminal. Overall, the design balances correctness, readability, and user experience, providing a robust yet developer-friendly tool for analyzing Lua source code.

5. Error Handling

Error handling in the Lua lexer was added to provide feedback and continue processing files with errors. A **catch-all rule using the dot (.) operator** is placed at the end of the Rules section to capture any character that does not match the defined Lua syntax, ensuring that unexpected input is flagged instead of ignored. When a string or block comment reaches the end of the file (<<EOF>>) without being properly closed, the print_error() function reports an **"Unterminated" message** along with the specific line number, helping developers quickly locate the issue. Similarly, invalid characters are reported as "Unknown character" errors in red text, but the **lexer continues scanning so that multiple issues can be detected in a single run**. This approach balances strict syntax enforcement with resilience, making the lexer both informative and robust during analysis.

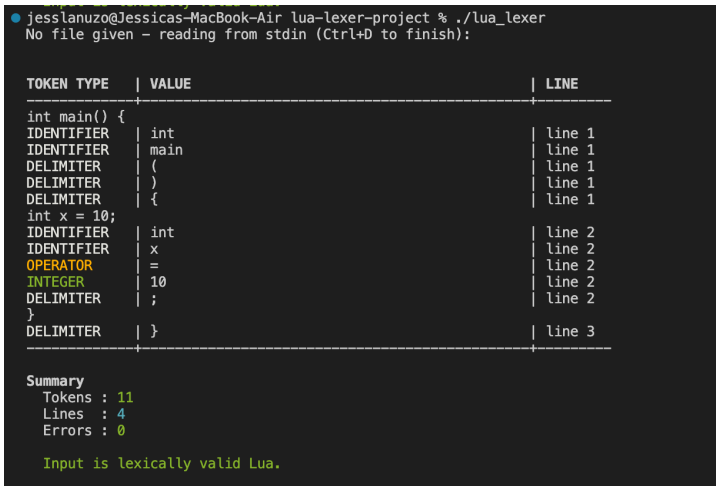
6. Testing & Sample Output

Test Case 1: Lexically Valid Code

Input:

```
int main() {
  int x = 10;
}
```

Expected Output: (Line count includes EOF (Ctrl+D), so it shows 4)



```
jesslanuzo@Jessicas-MacBook-Air lua-lexer-project % ./lua_lexer
No file given - reading from stdin (Ctrl+D to finish):

TOKEN TYPE   | VALUE                                          | LINE
-------------+------------------------------------------------+---------
int main() {
IDENTIFIER   | int                                            | line 1
IDENTIFIER   | main                                           | line 1
DELIMITER    | (                                              | line 1
DELIMITER    | )                                              | line 1
DELIMITER    | {                                              | line 1
int x = 10;
IDENTIFIER   | int                                            | line 2
IDENTIFIER   | x                                              | line 2
OPERATOR     | =                                              | line 2
INTEGER      | 10                                             | line 2
DELIMITER    | ;                                              | line 2
}
DELIMITER    | }                                              | line 3
-------------+------------------------------------------------+---------

Summary
  Tokens : 11
  Lines  : 4
  Errors : 0

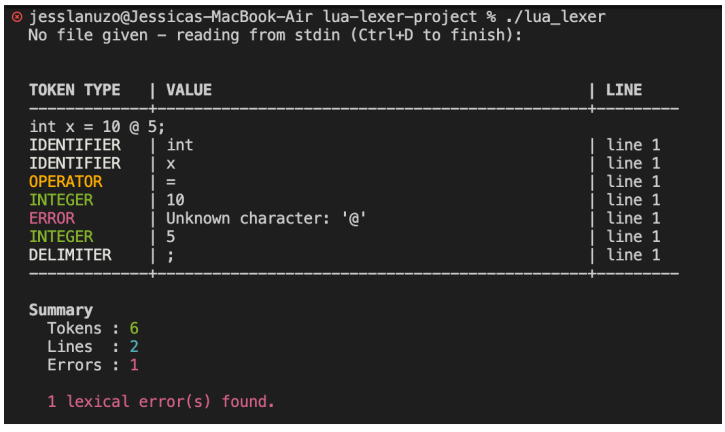
  Input is lexically valid Lua.
```

Test Case 2: Lexically Invalid Code

Input:

```
int x = 10 @ 5;
```

Expected Output: (Line count includes EOF (Ctrl+D), so it shows 2)



```
jesslanuzo@Jessicas-MacBook-Air lua-lexer-project % ./lua_lexer
No file given - reading from stdin (Ctrl+D to finish):

TOKEN TYPE   | VALUE                                          | LINE
-------------+------------------------------------------------+---------
int x = 10 @ 5;
IDENTIFIER   | int                                            | line 1
IDENTIFIER   | x                                              | line 1
OPERATOR     | =                                              | line 1
INTEGER      | 10                                             | line 1
ERROR        | Unknown character: '@'                         | line 1
INTEGER      | 5                                              | line 1
DELIMITER    | ;                                              | line 1
-------------+------------------------------------------------+---------

Summary
  Tokens : 6
  Lines  : 2
  Errors : 1

  1 lexical error(s) found.
```